

```
pip install pandas numpy matplotlib seaborn scikit-learn
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("car_data.csv")
df.head()
```

	Car_Name	Year	Selling_Price	Present_Price	Driven_kms	Fuel_Type
0	ritz	2014	3.35	5.59	27000	Petrol
1	sx4	2013	4.75	9.54	43000	Diesel
2	ciaz	2017	7.25	9.85	6900	Petrol
3	wagon r	2011	2.85	4.15	5200	Petrol
4	swift	2014	4.60	6.87	42450	Diesel

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
# Check null values
df.isnull().sum()

# Check dataset info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
```

```
Data columns (total 9 columns):
#      Column      Non-Null Count  Dtype
---  -
0     Car_Name    301 non-null    object
1     Year         301 non-null    int64
2     Selling_Price 301 non-null    float64
3     Present_Price 301 non-null    float64
4     Driven_kms     301 non-null    int64
5     Fuel_Type      301 non-null    object
6     Selling_type   301 non-null    object
7     Transmission  301 non-null    object
8     Owner         301 non-null    int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB
```

```
df = pd.get_dummies(df, drop_first=True)
```

```
X = df.drop('Selling_Price', axis=1)
y = df['Selling_Price']
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor()

model.fit(X_train, y_train)
```

```
▼ RandomForestRegressor ⓘ ?
RandomForestRegressor()
```

```
predictions = model.predict(X_test)
print(predictions[:5])
```

```
[0.4268  9.9664  4.9785  0.2014  7.4461]
```

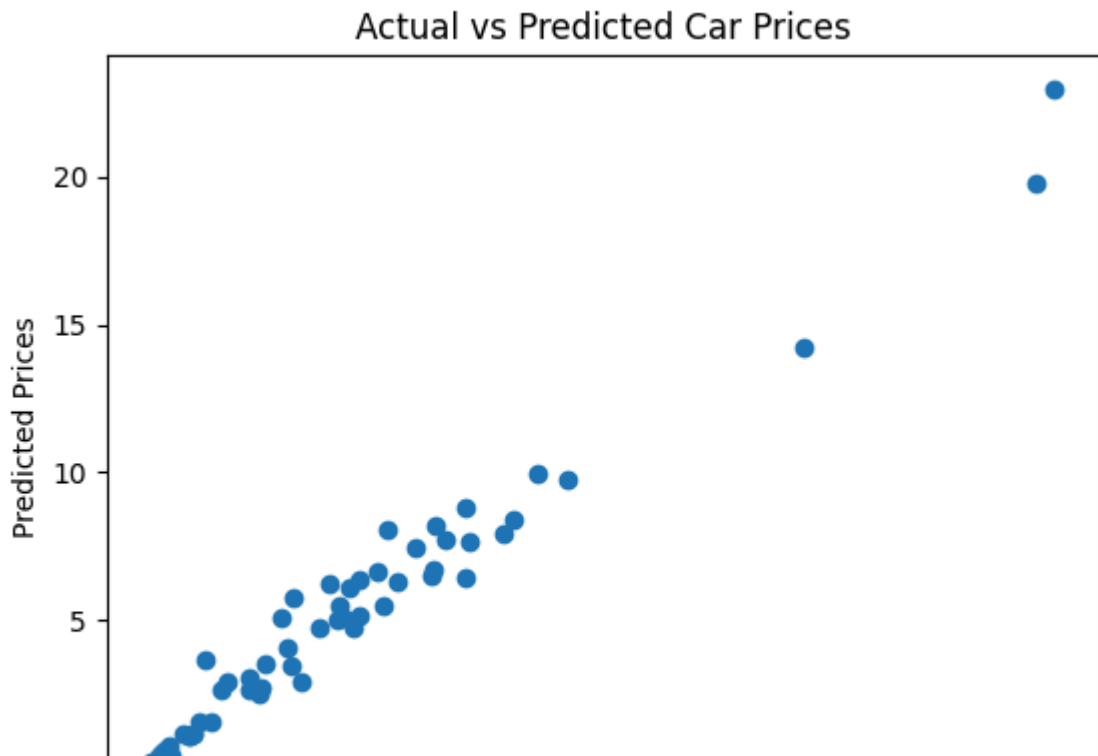
```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, predictions)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
r2 = r2_score(y_test, predictions)

print("MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

MAE: 0.6018737704918035
RMSE: 0.9239681808267688
R2 Score: 0.962939199635748

```
plt.scatter(y_test, predictions)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Car Prices")
plt.show()
```



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Step 2: Load Dataset
df = pd.read_csv("car data.csv") # change file name

# Step 3: Display Dataset
print(df.head())

# Step 4: Dataset Information
print(df.info())

# Step 5: Check Missing Values
print(df.isnull().sum())

# Step 6: Encode Categorical Columns
```

```

le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])

# Step 7: Feature Selection
X = df.drop("Selling_Price", axis=1)
y = df["Selling_Price"]

# Step 8: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 9: Train Model
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 10: Prediction
y_pred = model.predict(X_test)

# Step 11: Evaluation
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nModel Evaluation")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)

# Step 12: Actual vs Predicted Plot
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Car Prices")
plt.show()

# Step 13: Feature Importance
importance = model.feature_importances_

feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance
})

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

```

```
print(feature_importance)

# Step 14: Plot Feature Importance
plt.figure(figsize=(10,6))
sns.barplot(
    x='Importance',
    y='Feature',
    data=feature_importance
)

plt.title("Feature Importance")
plt.show()
```

